

Universidad Nacional de Ingeniería.
Recinto Universitario “Simón Bolívar”.



**DIRECCIÓN DEL ÁREA DE CONOCIMIENTO DE LA TECNOLOGÍA DE LA INFORMACIÓN Y LA
COMUNICACIÓN.**

INGENIERÍA DE SISTEMAS.

Programación II

Tema: Guía de Sistema de gestión de Producto

Integrantes: Abel Melquisedec Bermúdez Díaz

Maikeli Guivelki Rodríguez Centeno

David Alberto Reyes Castillo

Himland Ramses Oporta Alvarado

Grupo: 3M1-SIS-S.

Docente: Msc Abel Edmundo Marín Reyes.

Managua, Nicaragua 20 de marzo de 2026

```

using System.Text;
using System.Text.Json;

namespace FrmProductos
{
    3 referencias
    public partial class Form1 : Form
    {
        private List<Producto> listaProductos = new List<Producto>();
        private readonly string carpetaDatos = Path.Combine(Application.StartupPath, "Datos");
        private readonly string archivoJson;
        private readonly string archivoTexto;
        private readonly string archivoBinario;
        1 referencia
        public Form1()
        {
            InitializeComponent();
            archivoJson = Path.Combine(carpetaDatos, "productos.json");
            archivoTexto = Path.Combine(carpetaDatos, "productos.txt");
            archivoBinario = Path.Combine(carpetaDatos, "productos.dat");
        }

        1 referencia
        private void CrearCarpetaSiNoExiste()
        {
            if (!Directory.Exists(carpetaDatos))
            {
                Directory.CreateDirectory(carpetaDatos);
            }
        }
    }
}

```

“List Producto”

- Esta es una lista donde se guardarán todos los productos.

“Application.StartupPath”

- Es la ruta donde se ejecutará mi programa.

"Datos"

- Es la carpeta donde quiero guardar los archivos.

“archivoJson, archivoTexto, archivoBinario”

- Es la ruta completa de cada uno de los archivos.

“InitializeComponent(); “

- pone a trabajar todos los botones del formulario (botones, cajas de texto, etc.)
- archivoJson = Path.Combine(carpetaDatos, "productos.json");

“Path.Combine(...) “

- Es un método que une rutas de forma segura.

“carpetaDatos “

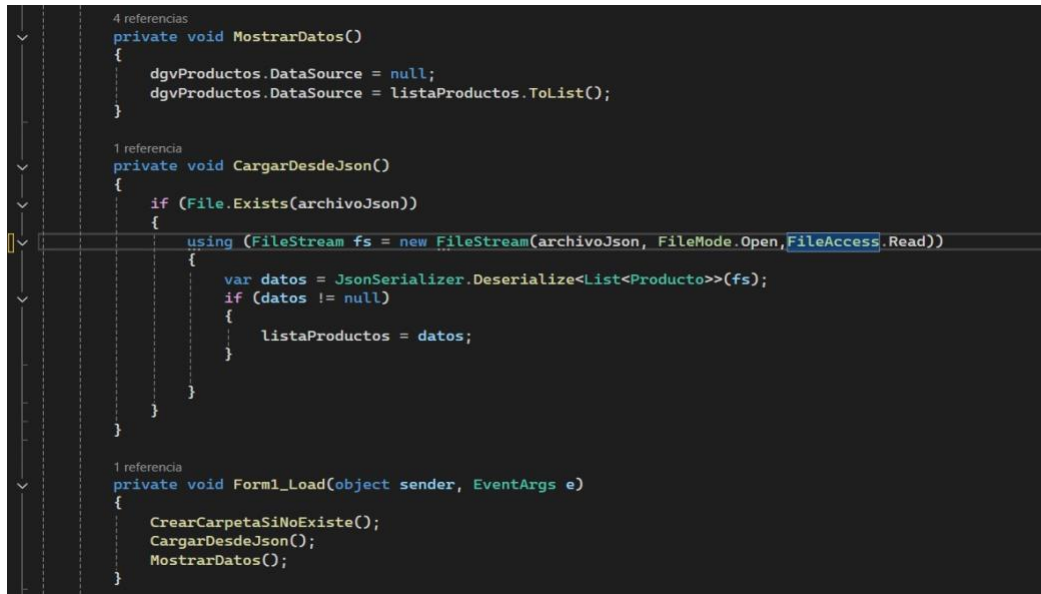
- Es una variable donde tengo la carpeta

“Private void crear carpeta sino existe ()”

- private: solo se puede usar dentro de esta clase.
- void: no devuelve ningún valor.
- CrearCarpetaSiNoExiste: nombre del método (describe lo que hace).

“if (!Directory.Exists(carpetaDatos)) “

- verifica si la carpeta existe.



```
4 referencias
private void MostrarDatos()
{
    dgvProductos.DataSource = null;
    dgvProductos.DataSource = listaProductos.ToList();
}

1 referencia
private void CargarDesdeJson()
{
    if (File.Exists(archivoJson))
    {
        using (FileStream fs = new FileStream(archivoJson, FileMode.Open, FileAccess.Read))
        {
            var datos = JsonSerializer.Deserialize<List<Producto>>(fs);
            if (datos != null)
            {
                listaProductos = datos;
            }
        }
    }
}

1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    CrearCarpetaSiNoExiste();
    CargarDesdeJson();
    MostrarDatos();
}
```

- dgvProductos es un **DataGridView** (una tabla en pantalla).
- DataSource es la fuente de datos que se muestra en esa tabla.
- dgvProductos.DataSource = null;
- Limpia la toda la tabla.
- Sirve para evitar errores o duplicados.

- `dgvProductos.DataSource = listaProductos.ToList();`
- Carga los datos de `listaProductos` en la tabla.
- `.ToList()` asegura que sea una lista compatible.
- `CargarDesdeJson()` → Carga todos los productos de un archivo `.json`.
- `if (File.Exists(archivoJson))`
- Revisa si el archivo si exista.
- Evita errores si no hay ningún archivo.
- `using (FileStream fs = new FileStream(...))`
- Abre el archivo en modo lectura.
- `FileMode.Open` → abre el archivo.
- `FileAccess.Read` → solo lectura.
- `using` → cierra el archivo automáticamente al terminar.
- `JsonSerializer.Deserialize<List<Producto>>(fs);`
- Convierte el JSON en una lista de objetos `Producto`
- `if (datos != null)`
- Verifica que sí se hayan cargado datos.
- `listaProductos = datos;`
- Guarda los datos en tu lista principal.

```

2 referencias
private bool ValidarCampos()
{
    if (string.IsNullOrWhiteSpace(txtId.Text) ||
        string.IsNullOrWhiteSpace(txtNombre.Text) ||
        string.IsNullOrWhiteSpace(txtPrecio.Text) ||
        string.IsNullOrWhiteSpace(txtCantidad.Text))
    {
        MessageBox.Show("Todos los campos son obligatorios.");
        return false;
    }

    if (!int.TryParse(txtId.Text, out _))
    {
        MessageBox.Show("El Id debe ser numérico.");
        return false;
    }

    if (!decimal.TryParse(txtPrecio.Text, out _))
    {
        MessageBox.Show("El precio debe ser numérico.");
        return false;
    }

    if (!int.TryParse(txtCantidad.Text, out _))
    {
        MessageBox.Show("La cantidad debe ser numérica.");
        return false;
    }

    return true;
}

```

“private bool ValidarCampos()”

- Es una función que valida los datos que el usuario escribe en los TextBox.
- Si algo está mal = devuelve `false`
- Si todo está bien = devuelve `verdadero`
- Validar los campos vacíos

**“if (string.IsNullOrEmpty(txtId.Text) ||
string.IsNullOrEmpty(txtNombre.Text) ||
string.IsNullOrEmpty(txtPrecio.Text) ||
string.IsNullOrEmpty(txtCantidad.Text))”**

- `IsNullOrEmpty` revisa si el campo:
- Está vacío
- Tiene espacios
- Es null

**“MessageBox.Show("Todos los campos son obligatorios.");
return false;”**

- Muestra un mensaje para el proceso

if (!int.TryParse(txtId.Text, out _))

- Intenta convertir el texto a número entero (int)
- `TryParse`:

Si puede = devuelve verdadero

Si no = devuelve falso

```
1 referencia
private Producto ObtenerProductoDesdeFormulario()
{
    return new Producto
    {
        Id = int.Parse(txtId.Text),
        Nombre = txtNombre.Text,
        Precio = decimal.Parse(txtPrecio.Text),
        Cantidad = int.Parse(txtCantidad.Text)
    };
}

3 referencias
private void GuardarEnJson()
{
    using (FileStream fs = new FileStream(archivoJson, FileMode.Create, FileAccess.Write))
    {
        JsonSerializer.Serialize(fs, listaProductos, new JsonSerializerOptions
        {
            WriteIndented = true
        });
    }
}

3 referencias
private void LimpiarCampos()
{
    txtId.Clear();
    txtNombre.Clear();
    txtPrecio.Clear();
    txtCantidad.Clear();
    txtId.Focus();
}
```

“private Producto ObtenerProductoDesdeFormulario()
{
 return new Producto
 {
 Id = int.Parse(txtId.Text),
 Nombre = txtNombre.Text,
 Precio = decimal.Parse(txtPrecio.Text),
 Cantidad = int.Parse(txtCantidad.Text)
 };
}”

- Este método **crea un objeto** Producto con los datos que el usuario escribió en los TextBox. Crea un **nuevo objeto** de la clase Producto

“return new Producto”

- Crea un **nuevo objeto** de la clase Producto

“Id = int.Parse(txtId.Text)”

- Convierte el texto a número entero

“FileStream

new FileStream(archivoJson, FileMode.Create, FileAccess.Write)”

- Abre o crea el archivo

“Create:”

Si existe lo sobrescribe

Si no existelo crea

“Write:”

Solo escribe en el archivo

“using (...)”

- Asegura que el archivo se cierre automáticamente
- Evita errores o archivos bloqueado

“JsonSerializer.Serialize(...)”

Convierte la lista (listaProductos) a formato JSON

WriteIndented = verdadero

LimpiarCampos()

```
private void LimpiarCampos()
{
    txtId.Clear();
    txtNombre.Clear();
    txtPrecio.Clear();
    txtCantidad.Clear();
    txtId.Focus();
}
```

ese código limpia todos los campos del formulario

Clear()

Borra el contenido del TextBox

txtId.Focus()

Coloca el cursor otra vez en el campo ID

```
1 referencia
private void btnAgregar_Click(object sender, EventArgs e)
{
    if (!ValidarCampos()) return;
    int id = int.Parse(txtId.Text);
    if (listaProductos.Any(p => p.Id == id))
    {
        MessageBox.Show("Ya existe un producto con ese Id.");
        return;
    }
    Producto nuevo = ObtenerProductoDesdeFormulario();
    listaProductos.Add(nuevo);
    GuardarEnJson();
    MostrarDatos();
    LimpiarCampos();

    MessageBox.Show("Producto agregado correctamente.");
}
```

private void btnAgregar_Click(object sender, EventArgs e)

Este método se ejecuta cuando haces clic en el botón.

Validar campos

if (!ValidarCampos()) return;

Llama al método que ya hiciste (ValidarCampos)

Si algo está mal:

return; detiene todo

Obtener el ID

int id = int.Parse(txtId.Text);

Convierte el texto a número para poder compararlo

Verificar si el ID ya existe

```
if (listaProductos.Any(p => p.Id == id))
```

Any revisa si ya existe un producto con ese ID

`p => p.Id == id` significa: “buscar un producto donde el Id sea igual al que ingresé”

Si en dado caso ya existe el ID:

```
MessageBox.Show("Ya existe un producto con ese Id.");
```

```
return;
```

Muestra mensaje

Detiene el proceso

Evita que se repita el mismo ID

agregar a la lista

```
listaProductos.Add(nuevo);
```

 Guarda el producto en una memoria

Guardar en JSON

```
GuardarEnJson();
```

 Guarda todos los productos en archivo

Mostrar datos

```
MostrarDatos();
```

 Actualiza la tabla o DataGridView

Limpiar campos

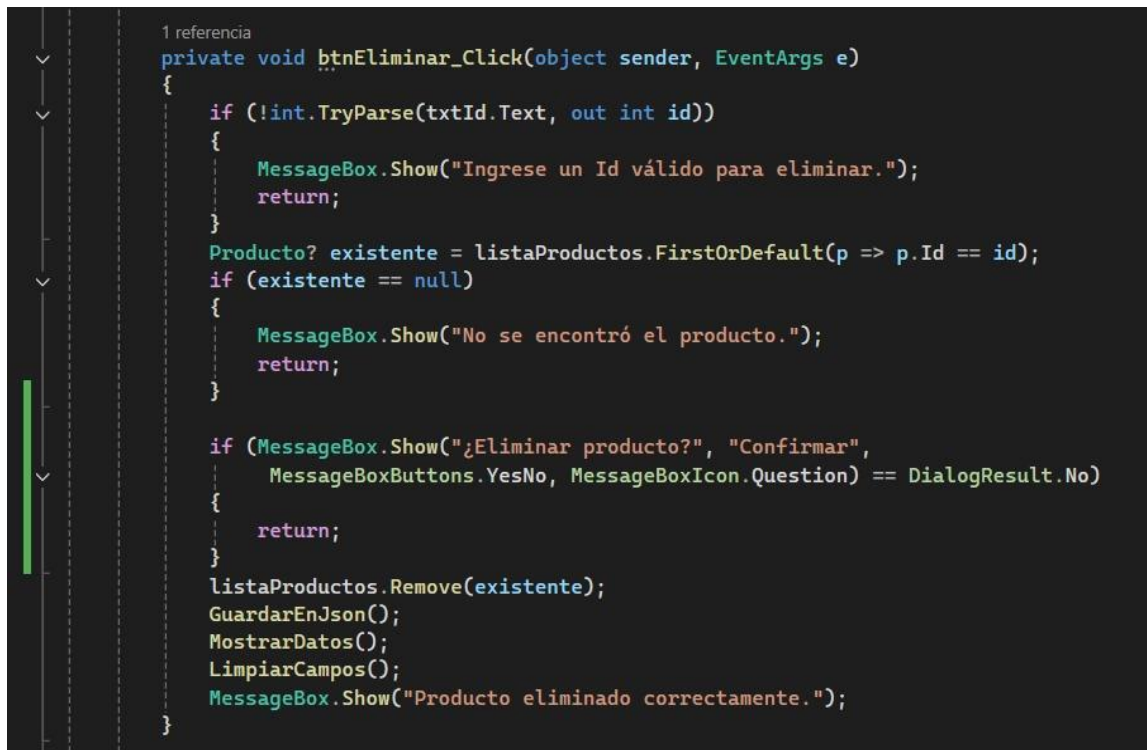
```
LimpiarCampos();
```

 Deja el formulario limpio y listo para otro registro

Mensaje final

```
MessageBox.Show("Producto agregado correctamente.");
```

 Confirma que todo salió bien



```

1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    if (!int.TryParse(txtId.Text, out int id))
    {
        MessageBox.Show("Ingrese un Id válido para eliminar.");
        return;
    }
    Producto? existente = listaProductos.FirstOrDefault(p => p.Id == id);
    if (existente == null)
    {
        MessageBox.Show("No se encontró el producto.");
        return;
    }

    if (MessageBox.Show("¿Eliminar producto?", "Confirmar",
        MessageBoxButtons.YesNo, MessageBoxIcon.Question) == DialogResult.No)
    {
        return;
    }
    listaProductos.Remove(existente);
    GuardarEnJson();
    MostrarDatos();
    LimpiarCampos();
    MessageBox.Show("Producto eliminado correctamente.");
}

```

“private void btnEliminar_Click(object sender, EventArgs e)”

- Esta parte del código se ejecuta cuando se le da click al botón de eliminar

“if (!int.TryParse(txtId.Text, out int id))”

- Acá convierte lo que se escribe en texto a un entero o sea un numero (Ejemplo: letras, vacío, etc)

“MessageBox.Show("Ingrese un Id válido para eliminar.");”

return;”

- Si nosotros ingresamos un texto en vez de un numero lo que va generar es un error y esta línea de código indica que se debe volver a escribir lo que se pide un ID, esto nos dice un número.

“Producto? existente = listaProductos.FirstOrDefault(p => p.Id == id);”

“if (existente == null)”

“MessageBox.Show("No se encontró el producto.");

return;”

- Esta parte del código lo que hace es buscar en la lista un producto que se haya ingresado con ese ID, sino lo encuentra va retornar que no ha encontrado ese producto, mientras que si se ingrese el ID correcto va encontrar el ID y lo va devolver.

“if (MessageBox.Show("¿Eliminar producto?", "Confirmar",

**MessageBoxButtons.YesNo, MessageBoxIcon.Question) ==
DialogResult.No)**

{

return;

}”

- Acá le aparecerá una ventana de confirmación, Si hace click en No se cancela todo y sigue corriendo el programa.

“listaProductos.Remove(existente);”

- De haber tocado en la confirmación el “SI” el producto que ingreso se va eliminar de la lista

“GuardarEnJson();”

- Actualiza el archivo JSON.

“MostrarDatos();”

- Refresca lo que es la tabla

“LimpiarCampos();”

- Limpia los textbox

```
1 referencia
private void btnEditar_Click(object sender, EventArgs e)
{
    if (!ValidarCampos()) return;
    int id = int.Parse(txtId.Text);
    Producto? existente = listaProductos.FirstOrDefault(p => p.Id == id);

    if (existente == null)
    {
        MessageBox.Show("No se encontró el producto.");
        return;
    }
    existente.Nombre = txtNombre.Text;
    existente.Precio = decimal.Parse(txtPrecio.Text);
    existente.Cantidad = int.Parse(txtCantidad.Text);
    GuardarEnJson();
    MostrarDatos();
    LimpiarCampos();
    MessageBox.Show("Producto editado correctamente.");
}
```

“private void btnEditar_Click(object sender, EventArgs e)”

- Así como en el método anterior este se ejecuta cuando se da click al botón de “EDITAR”.

“if (!ValidarCampos()) return;”

- Verifica que todos los campos que se van a ingresar sean correctos, sin embargo, si se pone algo que no está simplemente se detiene.

“int id = int.Parse(txtId.Text);”

- Acá convierte de texto a un número.

“Producto? existente = listaProductos.FirstOrDefault(p => p.Id == id);”

- (Ver página 10)

“if (existente == null)

{

MessageBox.Show("No se encontró el producto.");

return;

}”

- (Ver pagina 10)

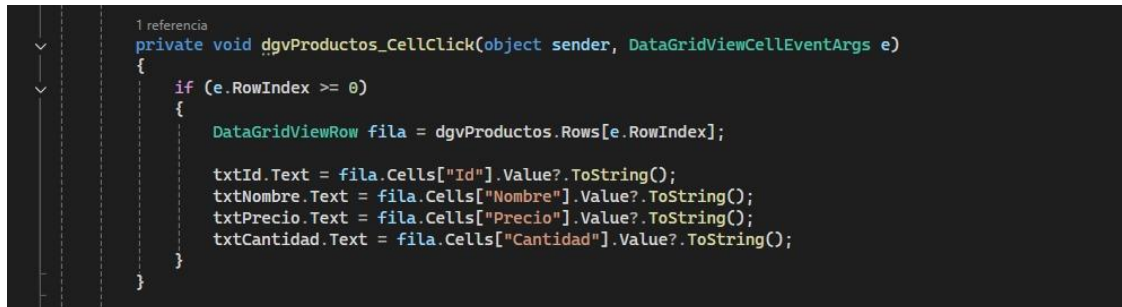
“existente.Nombre = txtNombre.Text;

existente.Precio = decimal.Parse(txtPrecio.Text);

existente.Cantidad = int.Parse(txtCantidad.Text);”

- Lo que hará es que editara directamente el producto que se ha encontrado, no crea uno nuevo.

Ahora procedemos a ubicarnos en el datagridview y generamos un evento



```
1 referencia
private void dgvProductos_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow fila = dgvProductos.Rows[e.RowIndex];

        txtId.Text = fila.Cells["Id"].Value?.ToString();
        txtNombre.Text = fila.Cells["Nombre"].Value?.ToString();
        txtPrecio.Text = fila.Cells["Precio"].Value?.ToString();
        txtCantidad.Text = fila.Cells["Cantidad"].Value?.ToString();
    }
}
```

“private void dgvProductos_CellClick(object sender, DataGridViewCellEventArgs e)”

- Este método se ejecuta cuando haces clic en una fila de la tabla (DataGridView) y Carga los datos de la fila seleccionada en los TextBox. Nos aseguramos que fuera en una fila real, guardamos la fila donde hicimos click. Luego de esto pasamos los datos a los TextBox

Y ahora creamos otro evento para el botón de ExportarTexto y copiamos el siguiente código:

```
1 referencia
private void btnExportarTexto_Click(object sender, EventArgs e)
{
    using (FileStream fs = new FileStream(archivoTexto, FileMode.Create, FileAccess.Write))
    using (StreamWriter writer = new StreamWriter(fs))
    {
        writer.WriteLine("REPORTE DE PRODUCTOS");
        writer.WriteLine("-----");

        foreach (var producto in listaProductos)
        {
            writer.WriteLine($"Id: {producto.Id}");
            writer.WriteLine($"Nombre: {producto.Nombre}");
            writer.WriteLine($"Precio: {producto.Precio}");
            writer.WriteLine($"Cantidad: {producto.Cantidad}");
            writer.WriteLine("-----");
        }
    }

    MessageBox.Show("Archivo de texto exportado correctamente.");
}
```

Este metodo que observamos aquí se ejecuta cuando haces clic en Exportar a Texto, este crea un archivo .txt con todos los productos.

“using (FileStream fs = new FileStream(archivoTexto, FileMode.Create, FileAccess.Write)) using (StreamWriter writer = new StreamWriter(fs))”

Esta parte nos permite crear el archive de texto y si ya existe, lo reemplaza (using) asegura que cierre correctamente.

En esta parte lo que hacemos es escribir cada producto (writer.WriteLine(\$"Id: {producto.Id}")); este escribe todos los datos y usa \$ para insertar valores (interpolación).

Ahora vamos a generar otro evento para el boton de vistaprevia y copiamos el siguiente código:

```
private void btnVistaPrevvia_Click(object sender, EventArgs e)
{
    using (MemoryStream ms = new MemoryStream())
    {
        string json = JsonSerializer.Serialize(listaProductos, new JsonSerializerOptions
        {
            WriteIndented = true
        });

        byte[] datos = Encoding.UTF8.GetBytes(json);
        ms.Write(datos, 0, datos.Length);

        ms.Position = 0;
    }
}
```

“btnVistaPrevia_Click”

Este método nos sirve para mostrar una vista previa de los productos en formato JSON dentro del programa (sin guardarlo en archivo). Convierte la lista de productos a JSON y nos lo muestra en pantalla (en el RichTextBox).

- En MemoryStream este crea un espacio en memoria y no en archivo físico, todo ocurre en RAM.
- String json, convierte la lista en Json y WriteIndent = true, nos lo muestra más bonito y ordenado.
- Convertir en bytes, nos convierte el texto JSON a formato binario, que nos referimos a bytes.
- ms.Write(datos, 0, datos.Length); Esta parte de aquí nos permite guardar los datos en el MemoryStream.
- Done observamos que dice ms.position = 0; este nos da paso a volver al inicio del stream para poder leerlo.
- Ya llegando al final de leer y mostrar, nos lee todo el contenido y nos lo muestra en el RichTextBox.

“btnRBinario_Click”

```
private void btnRBinario_Click(object sender, EventArgs e)
{
    using (FileStream fs = new FileStream(archivoBinario, FileMode.Create, FileAccess.Write))
    using (BinaryWriter writer = new BinaryWriter(fs))
    {
        writer.Write(listaProductos.Count);
        foreach (var producto in listaProductos)
        {
            writer.Write(producto.Id);
            writer.Write(producto.Nombre);
            writer.Write(producto.Precio);
            writer.Write(producto.Cantidad);
        }
    }
    MessageBox.Show("Respaldo binario generado correctamente.");
}
```

“private void btnRBinario_Click(object sender, EventArgs e)”

- Este método nos va a funcionar para guardar los productos en un archivo binario, cabe destacar que esto no es un texto legible solamente son datos en formato interno (más compactos)

“using (FileStream fs = new FileStream(archivoBinario, FileMode.Create, FileAccess.Write))

using (BinaryWriter writer = new BinaryWriter(fs))”

- Estas líneas nos dicen que se crea el archivo y así mismo se permite escribir datos en formato binario.

“writer.Write(listaProductos.Count);”

- Guarda cuantos productos hay y también nos va a servir para leer todo correctamente.

“foreach (var producto in listaProductos)”

- Recorre toda la lista que hemos construido.

“writer.Write(producto.Id);

writer.Write(producto.Nombre);

writer.Write(producto.Precio);

writer.Write(producto.Cantidad);”

- Acá le ponemos esta parte porque lo que hará será que guarde cada propiedad de lo que hemos definido en binario.

Url de Git Hub

[Gu-a-1/FrmProductos at main · DAReyesC/Gu-a-1](#)